MEDUSA — Status do Projeto e Plano de Conclusão

Data de referência: 19/09/2026

Projeto: MEDUSA — servidor local de IA

Status geral: Arquitetura em definição avançada; fase de engenharia de confiabilidade ainda em andamento.

1. Objetivo deste documento

Este documento registra o estado atual do projeto MEDUSA, separando:

- o que já foi definido ou concluído conceitualmente;

- o que permanece em aberto;

- as decisões técnicas que ainda dependem de validação prática;

- uma estimativa de esforço e prazo para chegar a uma primeira versão operacional estável.

> **Nota sobre tempo:** o histórico disponível registra as datas das sessões, mas não possui um contador confiável de horas efetivamente trabalhadas. Portanto, o tempo já utilizado é apresentado como **tempo de calendário decorrido**, e não como horas de trabalho faturáveis ou cronômetro de execução.

2. Visão do MEDUSA

A arquitetura atualmente considerada tem como base:

- **CPU:** AMD Ryzen 9 9950X

- **GPU:** NVIDIA RTX 5090 32 GB

- **Hypervisor:** Proxmox VE

- **Sistema da VM:** Ubuntu Server 26.04.1 LTS

- **GPU:** passthrough para a VM

- **Orquestração de aplicações:** Docker

- **Interface:** Open WebUI

- **Armazenamento externo:** NAS via SMB/CIFS

- **Volume de dados inicialmente previsto:** aproximadamente 500 GB, com possibilidade de expansão para 1 TB

- **Usuários simultâneos previstos:** aproximadamente 4–10

- **Uso:** múltiplos modelos locais/especializados, RAG e processamento de documentos

- **Requisito importante:** tarefas de ingestão/processamento em background devem reduzir sua atividade ao mínimo durante tarefas interativas e usar a capacidade máxima disponível quando não houver usuários interativos.

3. Trabalho realizado até agora

3.1 Definição do objetivo arquitetural — CONCLUÍDO

Foi estabelecido que o MEDUSA não deve ser tratado apenas como uma máquina executando um LLM, mas como uma plataforma local de IA com:

- virtualização;

- GPU dedicada por passthrough;

- containers;

- interface de utilização;
- acesso a armazenamento NAS;
- ingestão de documentos;
- modelos especializados;
- controle de concorrência;
- mecanismos de degradação e recuperação.

Estado: concluído conceitualmente.

3.2 Definição do hardware-alvo — CONCLUÍDO

O hardware de referência ficou estabelecido como:

Estado: definido; ainda requer validação operacional no host real.

3.3 Estratégia de armazenamento e NAS — DEFINIDO CONCEITUALMENTE

Foi definido que o NAS deverá participar do fluxo de dados sem transformar a rede/NAS em gargalo permanente.

Também foi estabelecido o requisito de que os diretórios utilizados pelo processamento possam ser selecionáveis no Open WebUI.

Estado: arquitetura conceitual definida; implementação e testes ainda pendentes.

3.4 Estratégia de ingestão em background — DEFINIDO CONCEITUALMENTE

Foi estabelecido um comportamento adaptativo:

- processamento máximo quando não houver usuários;
- redução agressiva durante tarefas interativas;
- possibilidade de pausa de trabalhos de background;
- retomada posterior;
- uso de filas persistentes;
- possibilidade de checkpoints/retries;
- isolamento de trabalhos problemáticos.

Estado: definido conceitualmente; falta implementar e validar.

4. Matriz de Falhas e Recuperação — principal avanço técnico recente

Foi iniciada a **Matriz de Falhas e Recuperação do MEDUSA**, que passa a orientar várias decisões de arquitetura.

Estados operacionais definidos

Faixas de RAM discutidas

Esses valores são **limiares de projeto**, não resultados de benchmark. Devem ser calibrados depois que a VM e os serviços estiverem instalados.

Swap

A estratégia discutida foi:

1. **ZRAM como primeira camada de amortecimento**;
2. disco/armazenamento como recurso de emergência;
3. evitar transformar swap em substituto de RAM;
4. observar impacto da pressão de memória sobre a inferência.

Filas

Foi considerada uma estratégia de filas persistentes com:

- retry inicial;
- retries progressivos;
- checkpoints;
- quarentena de tarefas problemáticas;
- retomada após recuperação.

Um exemplo discutido foi a progressão aproximada de retries em **30 s** $\rightarrow$ **5 min** $\rightarrow$ **30 min**.

Health checks

Foi definida a necessidade de health checks para:

- serviços Docker;
- camada de inferência;
- armazenamento;
- processamento de background;
- disponibilidade da interface;
- recuperação após falha.

Estado da matriz: versão inicial conceitual estabelecida; implementação e validação ainda pendentes.

5. Decisões ainda em aberto

5.1 Engine de inferência

Ainda precisa ser validada a escolha entre:

- **Ollama**
- **vLLM**
- **SGLang**

A decisão não deve ser feita apenas por desempenho bruto. Deve considerar:

- estabilidade;
- consumo de VRAM/RAM;
- concorrência;
- múltiplos modelos;
- facilidade de operação;
- integração com Open WebUI;
- recuperação após falha;
- observabilidade;
- comportamento sob pressão de memória;
- manutenção automatizada.

Estado: ABERTO.

5.2 Dimensionamento definitivo da VM

Ainda precisamos determinar empiricamente:

- RAM da VM;
- vCPU;
- reserva de memória;
- limites Docker;
- tamanho e comportamento da swap;
- espaço para cache;
- espaço para embeddings/RAG;
- espaço para temporários.

A matriz de falhas deve ser usada como requisito para esse dimensionamento.

Estado: ABERTO.

5.3 Concorrência e filas

Ainda falta definir e testar:

- número máximo de tarefas simultâneas;
- prioridade de requisições interativas;
- número de workers de ingestão;
- limites por serviço;
- política de backpressure;
- comportamento quando todos os recursos estiverem ocupados.

Estado: ABERTO.

5.4 RAG e ingestão documental

Ainda falta implementar/testar:

- pipeline de ingestão;
- parsing;
- chunking;
- embeddings;
- banco/vector store;
- atualização incremental;
- detecção de documentos alterados;
- remoção de documentos;
- reindexação;
- isolamento de documentos com erro.

Estado: ABERTO.

5.5 Integração NAS ↔ Open WebUI

Ainda falta validar:

- montagem SMB/CIFS;
- permissões;
- disponibilidade após reboot;
- reconexão;
- seleção de diretórios;
- desempenho;
- comportamento durante indisponibilidade do NAS;
- impacto sobre tarefas interativas.

Estado: ABERTO.

5.6 Observabilidade

Ainda falta definir e implementar métricas para:

- CPU;
- RAM;
- VRAM;
- temperatura;
- armazenamento;
- I/O;
- rede;
- filas;
- latência;
- throughput;
- erros;

- reinícios;

- estado dos containers.

Estado: ABERTO.

6. Etapas restantes

Fase 1 — Diagnóstico físico do servidor

Status: próxima etapa / parcialmente preparada.

Objetivo:

- coletar CPU;

- GPU;

- RAM;

- discos;

- PCIe;

- rede;

- BIOS/firmware;

- topologia;

- dispositivos relevantes para passthrough.

Saída esperada: relatório técnico do hardware.

Fase 2 — Preparação do Proxmox

- instalação/configuração;

- atualização;

- IOMMU;

- grupos PCIe;

- passthrough;

- bridges;

- storage;

- rede;

- parâmetros de boot;

- validação da RTX 5090 dentro da VM.

Fase 3 — Criação e dimensionamento da VM

- Ubuntu Server;

- vCPU;

- RAM;

- discos;

- ZRAM/swap;

- drivers NVIDIA;
- CUDA/runtime;
- passthrough;
- Docker.

Fase 4 — Plataforma Docker

Montagem dos serviços necessários e definição de:

- limites;
- reservas;
- volumes;
- redes;
- restart policies;
- health checks;
- logs;
- dependências.

Fase 5 — Engine de inferência

Executar testes comparáveis de Ollama, vLLM e/ou SGLang.

Métricas:

- tempo de carregamento;
- tokens/s;
- latência;
- VRAM;
- RAM;
- concorrência;
- estabilidade;
- comportamento sob pressão.

Fase 6 — Open WebUI

- instalação;
- integração com engine;
- modelos;
- permissões;
- diretórios;
- acesso ao NAS;
- usuários;
- testes de interação.

Fase 7 — NAS e ingestão

- SMB/CIFS;
- montagem persistente;
- ingestão;
- filas;
- workers;
- checkpoints;
- retries;
- quarentena.

Fase 8 — Matriz de Falhas e Recuperação

Transformar a matriz conceitual em automação real:

- monitor;
- estados;
- thresholds;
- throttling;
- pausa;
- retomada;
- recuperação;
- alertas;
- logs.

Fase 9 — RAG e modelos especializados

- embeddings;
- vector store;
- pipeline documental;
- modelos especializados;
- contexto jurídico de 32k;
- testes de recuperação;
- avaliação de qualidade.

Fase 10 — Testes de carga e falha

Testar:

- 1 usuário;
- múltiplos usuários;
- ingestão simultânea;
- pressão de RAM;
- pressão de VRAM;

- NAS indisponível;
- container reiniciado;
- engine reiniciada;
- reboot da VM;
- reboot do host;
- retomada de tarefas.

Fase 11 — Hardening e documentação final

- scripts de manutenção;
- backup das configurações;
- recuperação;
- procedimentos operacionais;
- documentação;
- checklist;
- baseline de desempenho.

7. Estimativa de tempo

Tempo já utilizado

O projeto aparece no histórico desta etapa desde **11/09/2026**, com a documentação sendo consolidada em **19/09/2026**.

Tempo de calendário transcorrido: aproximadamente 8 dias.

Não há um registro confiável de horas efetivamente trabalhadas. Portanto, não é correto transformar esses 8 dias em um número de horas de trabalho sem inventar dados.

8. Estimativa de esforço restante

A previsão abaixo é uma estimativa de engenharia para uma implementação cuidadosa, incluindo testes e documentação.

Estimativa total

Aproximadamente 36–70 horas de trabalho técnico restantes.

A faixa é deliberadamente ampla porque os maiores fatores de incerteza são:

1. comportamento real da RTX 5090 no passthrough;
2. dimensionamento de RAM;
3. escolha e comportamento da engine de inferência;
4. concorrência de 4–10 usuários;
5. pipeline de ingestão;
6. recuperação automática após falhas;
7. integração real com o NAS.

9. Previsão de calendário

Assumindo dedicação equivalente a:

Esses números representam **esforço técnico estimado**, não uma promessa de prazo. A etapa de testes pode revelar ajustes que aumentem ou reduzam o esforço.

10. Marco de conclusão recomendado

O MEDUSA deve ser considerado **operacionalmente concluído** somente quando:

- Proxmox estiver estável;
- GPU passthrough estiver validado;
- VM estiver dimensionada;
- Docker estiver funcionando;
- engine de inferência estiver escolhida e validada;
- Open WebUI estiver operacional;
- NAS estiver integrado;
- ingestão estiver funcionando;
- RAG estiver funcionando;
- concorrência estiver testada;
- matriz de falhas estiver automatizada;
- health checks estiverem ativos;
- recuperação automática estiver validada;
- testes de carga e falha estiverem concluídos;
- documentação operacional estiver pronta.

11. Estado consolidado

12. Próximo passo lógico

O próximo passo é executar o **diagnóstico automatizado do hardware do servidor**, gerar o relatório e usar os dados reais para fechar:

1. dimensionamento da VM;
2. estratégia de RAM/swap;
3. configuração de passthrough;
4. limites Docker;
5. capacidade de concorrência;
6. escolha entre Ollama, vLLM e SGLang.

A partir desse ponto, a arquitetura deixa de ser predominantemente teórica e passa a ser calibrada pelos recursos reais do servidor.

Documento de status — MEDUSA

Versão: 0.1

Data: 19/09/2026